

Effect of Associativity on Cache Performance

Implementation and experimental evaluation of a single-level set-associative data-cache simulator

Prepared by: Yehia, Jana, Mariam

Configuration: 64 KB cache · 64 B lines · Random replacement

Submission date: July 19, 2026

1. Cache Implementation and Verification

The project implements a real set-associative cache while keeping the provided memory generators and experiment harness unchanged. The simulator models tags, valid bits, dirty bits, random replacement, and a write-back/write-allocate policy.

1.1 Data organization and address mapping

The cache is stored as a two-dimensional `vector<vector<CacheLine>>`. The outer dimension represents sets, and each inner vector contains the ways in that set. Every line stores a tag, a valid bit, and a dirty bit. At the start of each run, `initCache(ways)` recalculates the number of sets, clears the cache state and write-back counter, and resets the replacement random-number stream.

$$\text{number of sets} = 64 \text{ KB} \div (64 \text{ B} \times \text{ways})$$

Address parsing uses portable arithmetic. The block number is `address / LINE_SIZE`; the offset is the remainder within the line; the set index is `block_number % num_sets`; and the remaining upper value is the tag. This produces the required `tag | set index | block offset` fields for all five associativities.

Hit path

Every way in the selected set is checked for a valid matching tag. A read returns a hit. A write also marks the line dirty because the changed data now exists only in the cache.

Miss path

An invalid way is filled first. If the set is full, `randReplace() % ways` chooses a victim. Replacing a dirty line counts one write-back before the new block is allocated.

1.2 Write policy

The simulator follows write-back with write-allocate. Every read or write miss loads the requested block. A write miss installs a dirty line, while a read miss installs a clean line. Memory traffic from replacement is counted only when a valid dirty line is evicted.

1.3 Evidence of testing

The final source was compiled as C++17 with strict warnings. An automated CTest regression test ran the full experiment, checked for exactly 30 data rows, and compared a fresh CSV byte-for-byte with `results.csv`.

PASS — 1/1 regression test passed and 0 tests failed. The fresh output matched every recorded ratio, access count, and write-back count.

Two results also act as independent checks. `memGen1` changes from continuous conflict misses at one way to two cold misses at two or more ways. `memGen5` gives the theoretical sequential result of 15 hits per 16 word accesses, or 93.75%.

2. Experimental Setup and Results

64 KB

CACHE CAPACITY

1,000,000

ACCESSES PER RUN

30

TOTAL RUNS

Cache size, line size, replacement policy, write policy, and address space are fixed. Only associativity changes. Each of the six unmodified generators is tested at 1, 2, 4, 8, and 16 ways from a cold cache. The address space is 64 MB, line size is 64 B, replacement is random, and writes use write-back with write-allocate.

2.1 Hit-ratio table

Generator	Access pattern	1-way	2-way	4-way	8-way	16-way
memGen1	K=2 same-set ping-pong	0.0000%	99.9998%	99.9998%	99.9998%	99.9998%
memGen2	Random over 64 MB DRAM	0.0972%	0.0996%	0.0983%	0.0969%	0.0983%
memGen3	K=8 same-set cyclic	0.0000%	0.7835%	17.6736%	99.9992%	99.9992%
memGen4	16 sets × K=4 conflict	0.0000%	14.2802%	99.9936%	99.9936%	99.9936%
memGen5	Sequential word stream	93.7500%	93.7500%	93.7500%	93.7500%	93.7500%
memGen6	Stack push/pop	99.9921%	99.9921%	99.9921%	99.9921%	99.9921%

Table 1. Measured hit ratios from results.csv.

2.2 Hit ratio versus number of ways

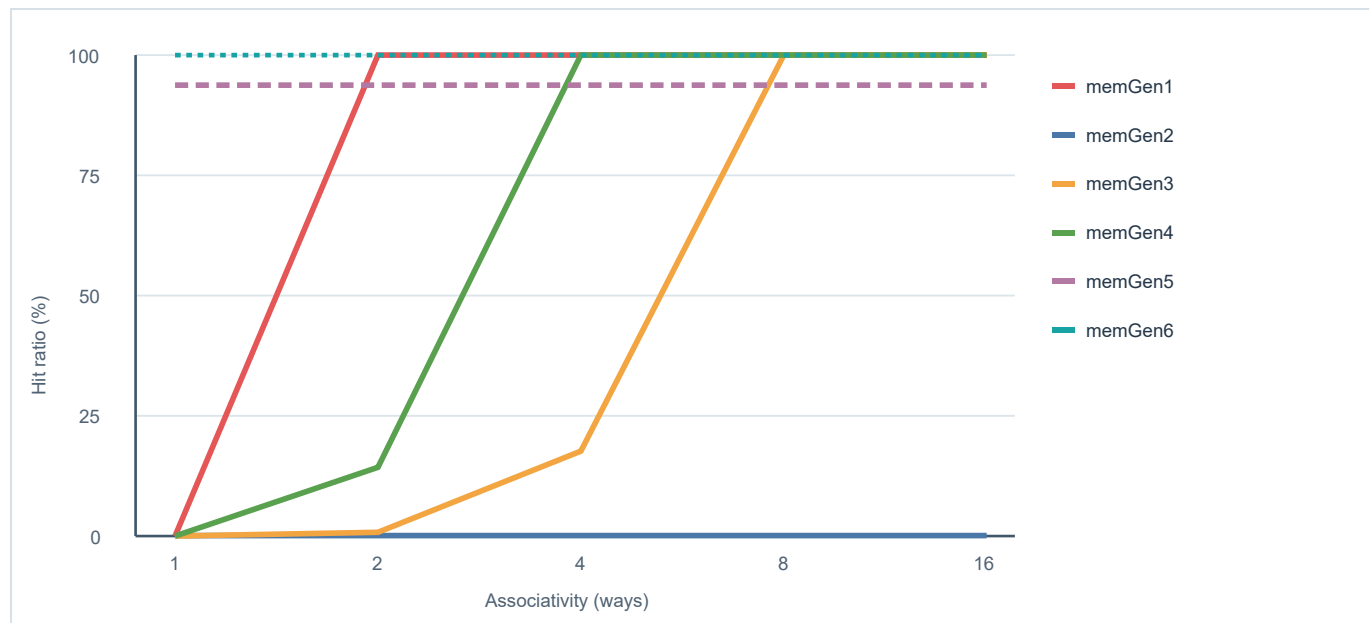


Figure 1. Conflict patterns rise when enough ways are available; locality and capacity patterns remain nearly flat.

3. Analysis of Access Patterns

3.1 Conflict-sensitive patterns

memGen1 alternates between two blocks mapped to the same set. A direct-mapped cache can keep only one, so each access removes the block needed next and the hit ratio is 0%. At two ways, both blocks remain after their first compulsory misses, producing 99.9998% hits. More ways give no further benefit.

memGen3 repeats the situation with eight conflicting blocks. With 1, 2, or 4 ways, random replacement often removes a block before reuse. This explains the rise from 0% to 0.7835% and 17.6736%. At eight ways, the entire group fits and hit ratio reaches 99.9992%.

memGen4 applies a four-block conflict across 16 sets. One way gives 0%, two ways gives 14.2802%, and four ways gives 99.9936%. The number of active sets increases traffic but does not change the four-way threshold.

Main conflict result: associativity removes thrashing when the ways reach the number of active same-set blocks: $K=2$ for memGen1, $K=4$ for memGen4, and $K=8$ for memGen3.

3.2 Patterns almost independent of associativity

memGen2 selects random addresses across 64 MB, about 1,024 times larger than the cache. Reuse before eviction is rare. More ways reorganize the same total capacity and cannot solve this capacity problem, so hit ratio stays near 0.1%.

memGen5 accesses consecutive four-byte words. One 64-byte line contains 16 words. The first word misses and the next 15 hit, giving $15/16 = 93.75\%$. Since the stream does not return to old lines, associativity cannot help. The exact value at all settings also verifies the line-size and address logic.

memGen6 uses a stack region of at most 16 KB. This is smaller than the 64 KB cache and has strong temporal locality, so the hit ratio remains 99.9921% for every associativity. The few misses are cold-cache line fills.

3.3 Capacity versus spatial locality

Capacity limitation

memGen2 needs more effective storage or more reuse. More ways cannot correct a working set much larger than the full cache.

Spatial locality

memGen5 benefits from the 64-byte line itself. Its hits come from nearby words already loaded by the first access.

4. Diminishing Returns and Conclusions

4.1 Where diminishing returns appear

More associativity is useful only while conflict misses are being removed. memGen1 reaches its useful limit at two ways, memGen4 at four ways, and memGen3 at eight ways. After these points, the working group already fits inside its set, so additional ways do not create new hits.

Higher associativity is not free in a real processor. More ways need more tag comparisons, a wider selection circuit, and usually more energy. A practical design should provide enough associativity for common same-set conflicts, without assuming that the largest possible number is always best.

4.2 Summary by generator

Generator	Main behavior	Useful associativity	Reason
memGen1	Two-line conflict	2-way	Both same-set lines fit together
memGen2	Random/capacity misses	No clear benefit	Working set is much larger than cache
memGen3	Eight-line conflict	8-way	All eight same-set lines fit
memGen4	Four-line conflicts in 16 sets	4-way	Each set holds its four-line group
memGen5	Sequential spatial locality	No clear benefit	15 of 16 words hit within each line
memGen6	Small stack working set	No clear benefit	Active data already fits

4.3 Final conclusion

The simulator produces the behavior expected from a correct set-associative cache. Associativity strongly improves conflict-bound workloads, but only until each set can hold the competing blocks. It does not solve a lack of total capacity and does not improve patterns whose performance already comes from spatial or temporal locality.

Two, four, and eight ways are the turning points for the K=2, K=4, and K=8 conflict generators. Above those points, hit ratio is unchanged. The sequential stream stays at its theoretical 93.75%, the stack stays near 100%, and random DRAM access stays near 0.1%. These stable results and the automated regression test support both the implementation and the conclusions.

Overall takeaway: increase associativity to control conflict misses, but stop when the same-set working group already fits. Beyond that point, hardware cost rises while performance does not.

Reproducibility note

The simulator resets generator, cache, and replacement state before every run. The repository includes the source, build and run configurations, regression test, testing evidence, and results.csv. Running the program without arguments reproduces the complete 30-run experiment.

GitHub: <https://github.com/mariamibrahim05/Project2->